

Module 2 — Workflows & Best Practices

Lesson 2.4

Debugging patterns

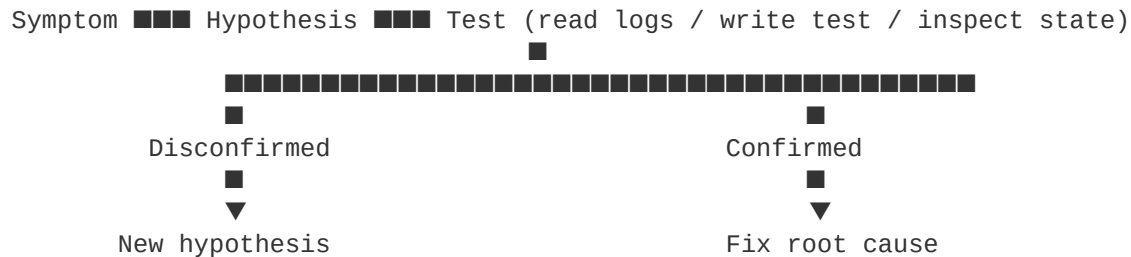
Debugging is hypothesis testing. Get Claude to state hypotheses explicitly, then prove or disprove them with data.

Mastering Claude Code — An E-Course for Power Users

Why it matters

The failure mode of LLM debugging is plausible-sounding speculation: "It might be a race condition, you could try X." That's not debugging — it's guessing. The patterns in this lesson force the conversation back to evidence.

The mental model



Concrete, evidence-bound. Claude can Grep, can Read, can construct a narrative — but the narrative is rooted in real lines, not vibes.

The minimal-repro pattern

"Write the smallest test case that reproduces this bug. Don't fix it yet. We'll add the test to the suite as a regression guard."

This achieves two things: it forces the bug into a deterministic shape, and it permanently locks in protection once fixed.

The "stop guessing" intervention

If Claude is suggesting third-thing-to-try without evidence:

"Stop suggesting fixes. State your current hypothesis in one sentence. State what evidence would confirm or disconfirm it. Get that evidence before suggesting anything else."

You're imposing discipline. It works.

The 5-whys variant

For root-cause hunts:

"Why did this fail? [...] OK, why did that happen? [...] Keep going until you reach something that's a true root cause vs. a symptom of something deeper. Then we'll fix at the root."

Claude is happy to fix symptoms (`raise from` becomes `try/except`). The 5-whys structure pushes past that.

When to delegate to a debugger subagent

When the search space is large (many files, many logs, many candidates):

```
"Spawn an Explore agent. Brief: find every place that could possibly write
to the `orders` table without going through `OrderService.create`.
We have an integrity violation that suggests a bypass somewhere. Report
file:line for each suspect site. Under 250 words."
```

You keep your context clean while the subagent does the noisy search.

Try it

Pick a real bug or a known-bad commit:

- 1 Open with the structured symptom prompt above.
- 2 Ask Claude for an explicit hypothesis before any fix.
- 3 Force it to gather evidence (read logs, write a repro test) before changing code.
- 4 After the fix, ask: "What's the root cause one level deeper than what we fixed?"

Gotchas

- **Claude will pattern-match to similar bugs in training data.** That's a hypothesis, not a diagnosis. Verify against your code.
- **"Let me try a different approach"** without explanation is a smell. Make Claude name what changed in the model.
- **Don't let it fix and verify in the same turn** for tricky bugs. Fix → run → report what happened → decide next step. Otherwise it conflates fixing with hoping.
- **Logs lie sometimes.** If the evidence and the hypothesis can't be reconciled, double-check the evidence (right env? right time window? right log level?).

Further reading

- 2.2 TDD with Claude — the regression-test pattern
- 4.3 Prompting subagents — briefing a debug helper

Next

→ 2.5 Git, commits, and PRs